


```
11:46 AM (38s) 3
> df_airports: pyspark.sql.connect.dataframe.DataFrame = [id: integer, ident: string ... 17 more fields]
> df_runways: pyspark.sql.connect.dataframe.DataFrame = [id: integer, airport_ref: integer ... 18 more fields]

11:46 AM (11s) 4 Python
# Cast columns
from pyspark.sql.functions import col

df_airports = df_airports \
    .withColumn("latitude_deg", col("latitude_deg").cast("double")) \
    .withColumn("longitude_deg", col("longitude_deg").cast("double")) \
    .withColumn("elevation_ft", col("elevation_ft").cast("int"))

df_runways = df_runways \
    .withColumn("length_ft", col("length_ft").cast("int")) \
    .withColumn("width_ft", col("width_ft").cast("int"))

display(df_airports)
display(df_runways)

> See performance (2)
> df_airports: pyspark.sql.connect.dataframe.DataFrame = [id: integer, ident: string ... 17 more fields]
> df_runways: pyspark.sql.connect.dataframe.DataFrame = [id: integer, airport_ref: integer ... 18 more fields]
```

practicalbigdata

File Edit View Run Help Python Last edit was 1 hour ago Run all Serverless Schedule Share

> See performance (2)

> df_airports: pyspark.sql.connect.dataframe.DataFrame = [id: integer, ident: string ... 17 more fields]

> df_runways: pyspark.sql.connect.dataframe.DataFrame = [id: integer, airport_ref: integer ... 18 more fields]

id	ident	type	name	latitude_deg	longitude_deg	elevation_ft	continent
1	6523 00A	heliport	Total RF Heliport	40.070985	-74.933689	11	NA
2	323361 00AA	small_airport	Aero B Ranch Airport	38.704022	-101.473911	3435	NA
3	6524 00AK	small_airport	Lowell Field	59.947733	-151.692524	450	NA
4	6525 00AL	small_airport	Epps Airpark	34.8647995	-86.77030182	820	NA
5	506791 00AN	small_airport	Katmai Lodge Airport	59.093287	-156.456699	80	NA
6	322127 00AS	small_airport	Fulton Airport	34.9428028	-97.8180194	1100	NA
7	6527 00AZ	small_airport	Cordes Airport	34.30559921	-112.1650009	3810	NA
8	6528 00CA	small_airport	Goldstone (GTS) Airport	35.35474	-116.885329	3038	NA
9	324424 00CL	small_airport	Williams Ag Airport	39.427188	-121.763427	87	NA
10	322658 00CN	heliport	Kitchen Creek Helibase Heliport	32.7273736	-116.4597417	3350	NA
11	6531 00FA	small_airport	Grass Patch Airport	28.64550018	-82.21900177	53	NA
12	6533 00FL	small_airport	River Oak Airport	27.23089981	-80.96920013	35	NA
13	6534 00GA	small_airport	Lt World Airport	33.76750183	-84.06829834	700	NA
14	6535 00GE	heliport	Caffrey Heliport	33.887982	-84.736983	957	NA
15							

10,000+ rows | Truncated data | 10.72s runtime Refreshed 1 hour ago

id	airport_ref	airport_ident	length_ft	width_ft	surface	lighted	closed	le_ident
1	269408	6523 00A	80	80	ASPH-G	1	0	H1
2	255155	6524 00AK	2500	40	GRVL	0	0	N
3	254165	6525 00AL	2100	90	TURF	0	0	1
4	506792	506791 00AN	4517	60	GVL	0	0	3
5	322128	322127 00AS	1450	60	Turf	0	0	1
6	257681	6527 00AZ	1700	60	GRAVEL	0	0	15
7	245528	6528 00CA	6000	80	ASPH	0	0	4
8	247972	6531 00FA	3200	100	TURF	0	0	8
9	250414	6533 00FL	4700	60	TURF	0	0	12
10	253429	6534 00GA	2600	80	TURF	0	0	9
11	265038	6535 00GE	125	95	ASPH	1	0	H1
12	265039	6536 00HI	1155	45	ASPH-G	0	0	H1
13	246648	6537 00ID	3500	40	TURF	0	0	8
14	246649	6537 00ID	2900	40	TURF	0	0	11
15								

10,000+ rows | Truncated data | 10.72s runtime Refreshed 2 hours ago

10,000+ rows | Truncated data | 10.72s runtime

Refreshed 2 hours ago

11:48 AM (<1s)5Python

```
# Print schema
print("Airports Schema:")
df_airports.printSchema()

print("Runways Schema:")
df_runways.printSchema()

|-- id: integer (nullable = true)
|-- airport_ref: integer (nullable = true)
|-- airport_id: string (nullable = true)
|-- length_ft: integer (nullable = true)
|-- width_ft: integer (nullable = true)
|-- surface: string (nullable = true)
|-- lighted: integer (nullable = true)
|-- closed: integer (nullable = true)
|-- le_ident: string (nullable = true)
|-- le_latitude_deg: double (nullable = true)
|-- le_longitude_deg: double (nullable = true)
|-- le_elevation_ft: integer (nullable = true)
|-- le_heading_degT: double (nullable = true)
|-- le_displaced_threshold_ft: integer (nullable = true)
|-- be_ident: string (nullable = true)
```

11:48 AM (1s)6

```
# Count rows
airports_count = df_airports.count()
runways_count = df_runways.count()

print("Number of rows in df_airports:", airports_count)
print("Number of rows in df_runways:", runways_count)
```

> [See performance \(2\)](#)

Number of rows in df_airports: 85324
Number of rows in df_runways: 47884

PART B - Join and Transform

11:53 AM (2s)8

```
# Inner join airports and runways
df_combined = df_airports.join(
    df_runways,
    df_airports.id == df_runways.airport_id,
    "inner"
)
```

> [See performance \(1\)](#)

> df_combined: pyspark.sql.connect.dataframe.DataFrame = [id: integer, ident: string ... 37 more fields]

	id	ident	type	name	latitude_deg	longitude_deg	elevation_ft	continent
1	6523	00A	heliport	Total RF Heliport	40.070985	-74.933689	11	NA
2	6524	00AK	small_airport	Lowell Field	59.947733	-151.692524	450	NA
3	6525	00AL	small_airport	Epps Airpark	34.8647995	-86.77030182	820	NA
4	506791	00AN	small_airport	Katmai Lodge Airport	59.093287	-156.456699	80	NA
5	322127	00AS	small_airport	Fulton Airport	34.9428028	-97.8180194	1100	NA
6	6527	00AZ	small_airport	Cordes Airport	34.30559921	-112.1650009	3810	NA
7	6528	00CA	small_airport	Goldstone (GTS) Airport	35.35474	-116.885329	3038	NA
8	6531	00FA	small_airport	Grass Patch Airport	28.64550018	-82.21900177	53	NA
9	6533	00FL	small_airport	River Oak Airport	27.23089981	-80.96920013	35	NA
10	6534	00GA	small_airport	Lt World Airport	33.76750183	-84.06829834	700	NA
11	6535	00GE	heliport	Caffrey Heliport	33.887982	-84.736983	957	NA
12	6536	00HI	heliport	Kaupulehu Heliport	19.832881	-155.978347	43	OC
13	6537	00ID	small_airport	Delta Shores Airport	48.14530182	-116.2139969	2064	NA
14	6539	00IL	small_airport	Hammer Airport	41.978401	-89.560402	840	NA
15								

```
11:54 AM (<1s) 9 Python
# Check schema
df_combined.printSchema()

|-- id: integer (nullable = true)
|-- airport_ref: integer (nullable = true)
|-- airport_ident: string (nullable = true)
|-- length_ft: integer (nullable = true)
|-- width_ft: integer (nullable = true)
|-- surface: string (nullable = true)
|-- lighted: integer (nullable = true)
|-- closed: integer (nullable = true)
|-- le_ident: string (nullable = true)
|-- le_latitude_deg: double (nullable = true)
|-- le_longitude_deg: double (nullable = true)
|-- le_elevation_ft: integer (nullable = true)
|-- le_heading_degT: double (nullable = true)
|-- le_displaced_threshold_ft: integer (nullable = true)
|-- he_ident: string (nullable = true)
|-- he_latitude_deg: double (nullable = true)
|-- he_longitude_deg: double (nullable = true)
|-- he_elevation_ft: integer (nullable = true)
|-- he_heading_degT: double (nullable = true)
|-- he_displaced_threshold_ft: integer (nullable = true)
```

```
11:54 AM (1s) 10
# Row count
print("Number of rows in df_combined:", df_combined.count())
> See performance \(1\)

Number of rows in df_combined: 47884
```

```
11:56 AM (1s) 11
from pyspark.sql.functions import col, when

# Add runway_category column
df_combined = df_combined.withColumn(
    "runway_category",
    when(col("length_ft") >= 10000, "Long")
    .when((col("length_ft") >= 5000) & (col("length_ft") < 10000), "Medium")
    .otherwise("Short")
)

# Preview result
display(df_combined.select("length_ft", "runway_category"))
> See performance \(1\)

df_combined: pyspark.sql.connect.dataframe.DataFrame = [id: integer, ident: string ... 38 more fields]
```

```
11:56 AM (1s) 11
> See performance \(1\)

df_combined: pyspark.sql.connect.dataframe.DataFrame = [id: integer, ident: string ... 38 more fields]
```

	length_ft	runway_category
1	80	Short
2	2500	Short
3	2100	Short
4	4517	Short
5	1450	Short
6	1700	Short
7	6000	Medium
8	3200	Short
9	4700	Short
10	2600	Short
11	125	Short
12	1155	Short
13	2900	Short
14	2500	Short
15	40	Short

Part c - Analysis Queries

11:58 AM (1s) 13 Python

```
from pyspark.sql.functions import count, col

# Top 10 countries with highest number of runways
top_10_countries = df_combined.groupBy("iso_country") \
    .agg(count("*").alias("runway_count")) \
    .orderBy(col("runway_count").desc()) \
    .limit(10)

# Display result
display(top_10_countries)
```

> [See performance \(1\)](#)

> top_10_countries: pyspark.sql.connect.dataframe.DataFrame = [iso_country: string, runway_count: long]

	iso_country	runway_count
1	US	26450
2	BR	5825
3	AU	2103

11:58 AM (1s) 13 Python

```
display(top_10_countries)
```

> [See performance \(1\)](#)

> top_10_countries: pyspark.sql.connect.dataframe.DataFrame = [iso_country: string, runway_count: long]

	iso_country	runway_count
1	US	26450
2	BR	5825
3	AU	2103
4	CA	1578
5	AR	761
6	FR	659
7	GB	600
8	DE	592
9	RU	489
10	ID	414

10 rows | 1.49s runtime Refreshed 1 hour ago

12:01 PM (1s) 14

```
from pyspark.sql.functions import avg, col

# Average runway length per continent
avg_runway_length = df_combined \
    .filter(col("length_ft").isNotNull()) \
    .groupBy("continent") \
    .agg(avg("length_ft").alias("avg_length_ft")) \
    .orderBy(col("avg_length_ft").desc())

# Display result
display(avg_runway_length)
```

> [See performance \(1\)](#)

> avg_runway_length: pyspark.sql.connect.dataframe.DataFrame = [continent: string, avg_length_ft: double]

	continent	avg_length_ft
1	AN	7768.857142857143
2	AS	6925.6469996647675
3	AF	6787.526448362721
4	EU	4103.151335311572
5	OC	3558.0401785714284
6	SA	2857.0891141239963

12:02 PM (1s) 15

```
from pyspark.sql.functions import col, count

# Airports in India with at least one lighted runway
india_lighted_runways = df_combined \
    .filter((col("iso_country") == "IN") & (col("lighted") == "1")) \
    .groupBy("name", "municipality") \
    .agg(count("*").alias("number_of_lighted_runways")) \
    .orderBy(col("number_of_lighted_runways").desc())

# Display result
display(india_lighted_runways)
```

> [See performance \(1\)](#)

india_lighted_runways: pyspark.sql.connect.dataframe.DataFrame = [name: string, municipality: string ... 1 more field]

	name	municipality	number_of_lighted_runways
1	Indira Gandhi International Airport	New Delhi	3
2	Navi Mumbai International Airport	Navi Mumbai	2
3	Kempegowda International Airport Bengaluru	Bengaluru	2
4	Mangaluru International Airport	Mangaluru	2
5	Chennai International Airport	Chennai	2

12:03 PM (1s) 16 Python

```
from pyspark.sql.functions import max, col

# Top 5 airports with longest maximum runway length
top_5_airports = df_combined \
    .filter(col("length_ft").isNotNull()) \
    .groupBy("name", "iso_country") \
    .agg(max("length_ft").alias("max_runway_length")) \
    .orderBy(col("max_runway_length").desc()) \
    .limit(5)

# Display result
display(top_5_airports)
```

> [See performance \(1\)](#)

top_5_airports: pyspark.sql.connect.dataframe.DataFrame = [name: string, iso_country: string ... 1 more field]

	name	iso_country	max_runway_length
1	Gunflint Seaplane Base	US	30000
2	Libby Camps Seaplane Base	US	26000
3	Brookville Reservoir Seaplane Ba...	US	25000

Part D - Save as Parquet

12:08 PM (3s) 18: Cell 18

```
# Save df_combined as Parquet in Databricks Volume
# Drop duplicate 'id' column from runways to avoid COLUMN_ALREADY_EXISTS error
df_combined.drop(df_runways.id) \
    .write \
    .mode("overwrite") \
    .parquet("/Volumes/vita/dataengineering/data/df_combined_parquet")
```

> [See performance \(1\)](#)

12:09 PM (3s) 19 Python

```
# Read parquet back
df_check = spark.read.parquet(
    "/Volumes/vita/dataengineering/data/df_combined_parquet"
)

display(df_check)
print(df_check.count())
```

> [See performance \(2\)](#)

df_check: pyspark.sql.connect.dataframe.DataFrame = [id: integer, ident: string ... 37 more fields]